自己学会下五子棋的 AI 程序

项目介绍（面向非计算机背景的读者） · Eric · 2026 年 10 月

一页看懂

这个项目是一个**自己练会下五子棋的电脑程序**。它事先只知道"五子连成一线就赢"这条规则，没有人教它任何下法和套路。它靠自己和自己下上万盘棋，从每盘的输赢里总结经验，慢慢变强。这种让电脑"从输赢中自学"的方法叫**强化学习**，著名的围棋程序 AlphaGo 用的就是这一类方法。

9×9 小棋盘	15×15 标准棋盘	训练设备
90% 以上	**22%**	**个人笔记本**
对"会基本战术的规则程序"的胜率，从零练起，用时约 3 小时	已能赢下一部分对局，防守仍是短板，下一步加入"往前推演"的能力	全部训练在一台 MacBook 上完成，没有使用专业服务器

1 这个项目做什么

2016 年 AlphaGo 战胜围棋世界冠军。之后 DeepMind 公司又做出了 AlphaZero：它不看任何人类棋谱，只知道规则，完全靠自己和自己下棋练成了顶尖高手。AlphaZero 训练时用的是大型计算机集群。这个项目的目标是在**一台普通的个人电脑上**做一个缩小版，选的游戏是规则简单的五子棋。

强化学习可以这样理解：就像一个刚学棋的小孩，没有老师讲套路，每下完一盘只被告诉"你赢了"或"你输了"。下了几万盘之后，他会慢慢总结出"这样下容易赢、那样下容易输"。程序做的是同样的事，只是下棋的速度快得多。

2 程序是怎么学会下棋的

程序采用的学习方法叫 **DQN**（"深度 Q 网络"）。可以这样理解：程序看着当前的棋盘，给每个空位打一个"在这里下有多好"的分数，然后挑分数最高的位置落子。一开始这些分数是乱打的，随着一盘盘下棋，它根据输赢不断修正自己的打分。这种方法对电脑的要求比 AlphaZero 低得多，个人笔记本就能运行。

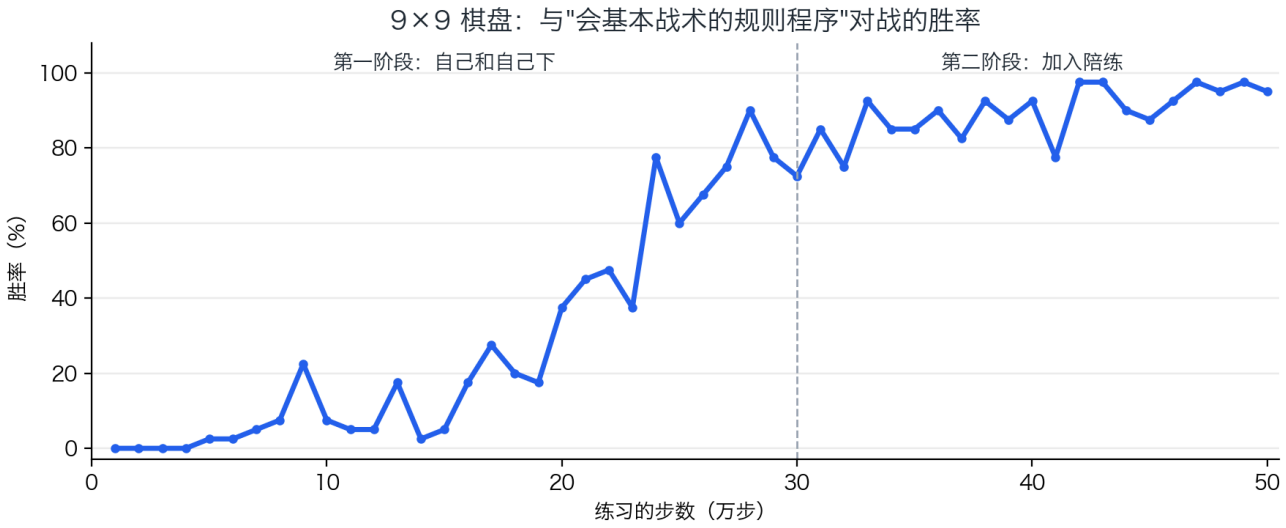
具体有以下几个设计：

1. **记分规则。**每盘结束时，赢的一方 +1，输的一方 −1。程序轮流执黑、执白和自己下，它会把"对手的好棋"记为"自己的坏结果"，因为下棋时对手的妙手对下棋的这一方来说正是坏消息。
2. **会"看棋形"的大脑。**程序的"大脑"是一个卷积神经网络，它像人眼一样逐块扫视棋盘，一个"活三"（三个子连成一线、两头都空着的棋形）不管出现在棋盘哪里都能认出来。
3. **一盘棋当八盘用。**把一盘棋旋转 90 度或翻个面，本质上还是同一个局面。利用这一点，每盘棋可以变成八份练习材料。

4. **用固定的"考官"衡量水平。**训练过程中定期让程序和两个固定的对手各下几十盘，看胜率：一个是**乱下的程序**（相当于及格线），一个是**会基本战术的规则程序**（会连子、会堵对方的四个子，相当于一个懂规则的初学者）。
5. **两个训练阶段。**第一阶段让程序自己和自己下。第二阶段加入"陪练"：一半的对局改为和规则程序下，让它多见识"对方已经连成四个子、必须去堵"这类自己和自己下时很少遇到的局面。
6. **自动检查。**程序配有 15 项自动检查。例如人为摆出一盘"黑棋连成五子获胜"的棋，检查输掉的白棋是否被正确扣分，保证每个环节都按设计运行。

3 9×9 棋盘：结果

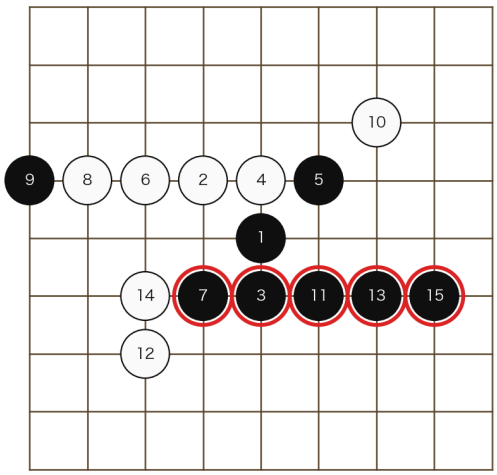
先在 9×9 的小棋盘上训练。小棋盘只有 81 个空位，学得快，适合验证方法是否有效。



程序与规则程序对战的胜率随练习量的变化。每个点是 40 盘对局的胜率。前 15 万步胜率很低，随后开始爬升；加入陪练后多数时候在 85% 以上。

阶段	练习量与用时	对乱下的程序	对规则程序
第一阶段：自己和自己下	30 万步，104 分钟	100%	75%
第二阶段：加入陪练	再练 20 万步，69 分钟	100%	90% 到 94%

"步"指程序在练习中落下的一颗棋子。表中成绩是正式测试 100 盘得出的，开局前两步随机摆放，避免每盘都下成一模一样的棋。第二阶段的程序与第一阶段的程序直接对战 100 盘，胜率 64%。



一局实战：程序执黑（先手），规则程序执白。数字是落子顺序。

这盘棋里发生了什么

白棋（规则程序）在上方横着连了四个子（第 2、4、6、8 手），黑棋在两端及时堵住（第 5、9 手），没让它连成五子。

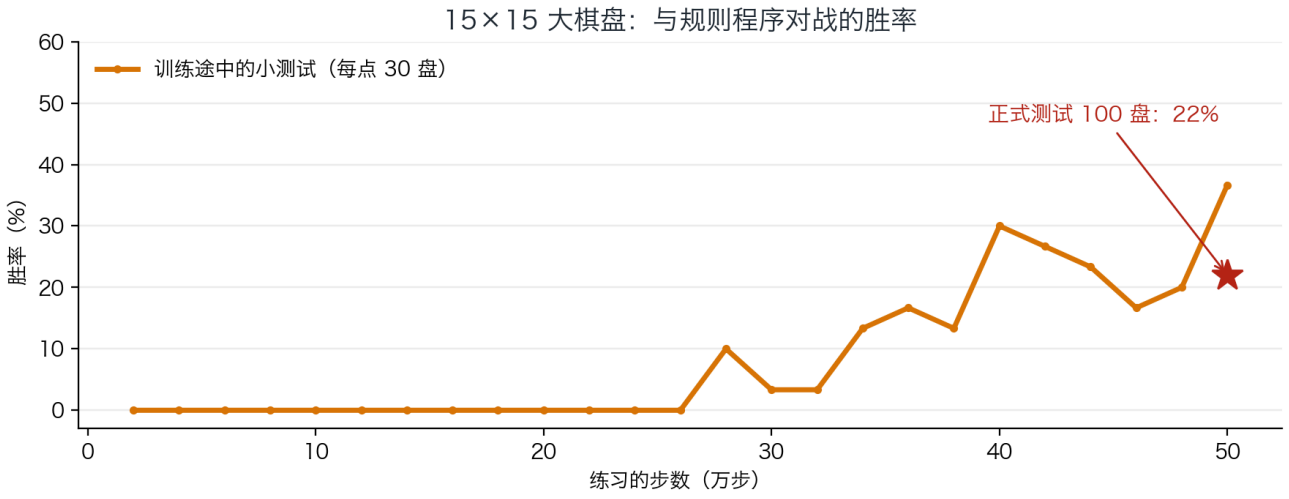
与此同时，黑棋在下面一行连出了一条两头都空着的四子线（第 3、7、11、13 手）。白棋只能堵住一头（第 14 手），黑棋从另一头落下第 15 手，连成五子获胜（红圈）。

程序同时会**防守**（堵住对方的四）和**进攻**（做出对方堵不住的棋形），而这些都没有人教过它。

训练中还能看到一个有意思的现象：程序先学会进攻，每盘棋变得很短（平均 14 步就结束）；之后学会了防守，双方你来我往，每盘棋又变长到 23 步左右。

4 15×15 标准棋盘：结果

15×15 是正式比赛用的棋盘，空位从 81 个增加到 225 个，可能出现的局面多了非常多，难度大很多。在这个棋盘上，程序"大脑"的最后一层也采用"看局部棋形"的结构，学会的规律在棋盘任何位置都通用。



15×15 棋盘上与规则程序对战的胜率。曲线上每个点只是训练途中 30 盘的小测试，波动较大；以训练结束后正式测试 100 盘的结果为准，即红色星号标出的 22%。

- **练习量：**50 万步，用时 5.3 小时。对乱下的程序胜率 100%。
- **对规则程序：**前 28 万步一盘没赢，之后开始有进步，正式测试 100 盘胜率 22%。
- **短板：**输掉的棋局大多是对方已经连成四个子，程序没有去堵。这说明"看一眼局面就落子、不往前多想"的方法，在大棋盘上已经接近它的上限，单纯增加训练时间意义不大。

5 下一步计划

人下棋时会"往前想几步"：这步下在这里，对手可能怎么应，接下来自己再怎么走。AlphaGo 和 AlphaZero 也是这样做的，这在技术上叫做**搜索**。目前的程序只有"直觉"，没有"推演"。下一步考虑两个方向：

- 保留现在训练好的程序作为"直觉"，再给它加上往前推演几步的能力。"对方已经四连、必须去堵"这类问题只需要往前看一步就能发现，预计能明显改善大棋盘上的表现。
- 在节省计算资源的设置下，尝试运行原版的 AlphaZero 方法，与现在的方法比较。

6 使用的工具

- **编程与训练：**编程语言 Python，深度学习工具 PyTorch。全部训练在一台个人 MacBook 上完成，使用了苹果芯片自带的图形加速功能。
- **代码基础：**项目建立在 Michael Hu 公开的 AlphaZero 学习项目之上（开源许可，允许自由使用和修改），复用了其中的五子棋规则部分。

> **术语小词典**
>
> **强化学习**
> 不给标准答案，只在做完之后告诉"好"或"不好"，让程序自己摸索出好的做法。
>
> **神经网络**
> 程序的"大脑"，一种可以从大量例子中总结规律的数学模型。
>
> **训练 / 练习步数**
> 程序通过下棋积累经验的过程。一步指落下一颗棋子。在 9×9 棋盘上，30 万步大约是一万两千多盘棋。
>
> **规则程序**
> 本项目里用作陪练和考官的对手，按固定规则下棋：能赢就赢，对方要连成五子就堵，否则尽量把自己的子连起来。